

HOOPPICKS

TEST

Test de conversion

ligne 1

ligne 2

Table des matières

Dans Word : clic droit sur la table puis « Mettre à jour les champs » pour afficher les numéros de page.

Introduction : comprendre Hoop Picks	3
À qui s'adresse ce document	3
Ce que fait l'application	3
Les trois types de paris	3
La cote et le ticket combiné.....	4
L'architecture générale.....	4
Le trajet d'une requête, de bout en bout.....	4
Les couches, dossier par dossier.....	4
Les technologies employées	5
Java et Spring Boot.....	5
L'injection de dépendances	5
JPA, Hibernate et PostgreSQL.....	6
Lombok.....	6
Les décisions d'architecture à connaître	6
Conventions de lecture.....	7

Introduction : comprendre HoopPicks

À qui s'adresse ce document

Ce document explique l'**intégralité du code backend** de HoopPicks, fichier par fichier et fonction par fonction. Il est écrit pour quelqu'un qui découvre le projet : ni son historique, ni ses conventions, ni forcément les technologies qu'il utilise.

Le seul prérequis est de savoir ce qu'est un programme et une fonction. Si vous avez déjà touché à React ou à du JavaScript, vous serez à l'aise ; si vous n'avez jamais écrit une ligne de Java ni entendu parler de Spring Boot, tout est expliqué au fur et à mesure, la première fois que la notion apparaît.

Le document est fait pour être lu dans l'ordre au moins une fois, puis consulté comme une référence : chaque fichier du projet a sa propre section, avec son code source complet reproduit juste après l'explication.

Ce que fait l'application

HoopPicks est une application de **pronostics sur les matchs de NBA**, jouée avec des **points virtuels**. Aucun argent réel n'est en jeu à aucun moment : chaque utilisateur reçoit 1000 points à l'inscription, les mise sur des matchs à venir, et gagne ou perd des points selon les résultats réels.

Le parcours typique d'un utilisateur ressemble à ceci :

1. Il crée un compte et reçoit 1000 points virtuels.
2. Il consulte la liste des matchs NBA à venir, avec pour chacun trois façons de parier.
3. Il clique sur les cotes qui l'intéressent : ses choix s'accumulent dans un « panier » (le *bet slip*).
4. Il choisit combien de points miser et valide son ticket. La mise est immédiatement débitée de son solde.
5. Quand les matchs se terminent, l'application récupère les scores réels, détermine automatiquement si le ticket est gagnant, et crédite les gains le cas échéant.
6. Il suit sa progression : classement général, ligues privées entre amis, badges débloqués, historique de ses paris.

Les trois types de paris

Trois « marchés » sont proposés sur chaque match, et ils reviennent constamment dans le code :

- **Moneyline** : le pari le plus simple. Qui gagne le match ? On mise sur l'équipe à domicile ou sur l'équipe visiteuse.
- **Spread** : un handicap destiné à équilibrer un match déséquilibré. Si le spread est de -5,5 pour l'équipe à domicile, elle doit gagner avec au moins 6 points d'écart pour que le pari passe. Miser sur l'autre côté (+5,5) est gagnant si l'équipe visiteuse gagne **ou** perd de 5 points ou moins.
- **Total** (aussi appelé *over/under*) : on ne parie pas sur le vainqueur mais sur le nombre total de points marqués par les deux équipes réunies. Si le total est fixé à 220,5, parier « plus » est gagnant si les deux équipes marquent 221 points ou plus au cumul.

La cote et le ticket combiné

Une **cote** est un multiplicateur. Une cote de 1,80 signifie qu'une mise de 100 points en rapporte 180 si le pari est gagnant (soit 80 points de bénéfice). Plus une issue est jugée probable, plus la cote est basse.

Un ticket peut contenir **plusieurs sélections** : c'est un **combiné**. Les cotes se multiplient — deux sélections à 1,80 donnent une cote totale de 3,24 — mais **toutes** les sélections doivent être gagnantes. Un combiné rapporte donc beaucoup plus, avec un risque bien supérieur.

L'architecture générale

Le projet est un **monorepo** : un seul dépôt Git qui contient deux applications distinctes.

Dossier	Rôle	Technologies
<code>backend/</code>	Le serveur : logique métier, base de données, appels aux APIs externes	Java 21, Spring Boot 4.1, JPA/Hibernate, PostgreSQL
<code>frontend/</code>	L'interface web visible par l'utilisateur	Next.js 16, React 19, TypeScript, Tailwind CSS v4

Les deux communiquent uniquement par **HTTP en JSON**. Le frontend n'accède jamais directement à la base de données : il appelle des URLs exposées par le backend (par exemple `GET /matches` pour récupérer la liste des matchs), et le backend lui répond avec du JSON.

Ce document couvre le **backend**. Un document séparé couvre le frontend.

Le trajet d'une requête, de bout en bout

Comprendre ce trajet permet de situer immédiatement n'importe quel fichier du projet. Prenons l'exemple d'un utilisateur qui ouvre la page des matchs :

7. **Le navigateur** demande la liste des matchs au backend.
8. **Le contrôleur** (`MatchController`) reçoit la requête sur l'URL `/matches`. C'est le point d'entrée : il ne contient presque pas de logique, il oriente.
9. **Le service** (quand il y en a un) applique la logique métier : calculs, règles, vérifications.
10. **Le repository** (`MatchRepository`) va chercher les données en base. C'est la seule couche qui parle à PostgreSQL.
11. Les données remontent sous forme d'**entités** (`Match`, `Team`), qui sont des objets Java reflétant les tables de la base.
12. Le contrôleur convertit ces entités en **DTOs** (`MatchDto`) — des objets pensés pour être envoyés au frontend, sans les champs sensibles ni la mécanique interne.
13. Spring Boot transforme automatiquement le DTO en **JSON** et le renvoie au navigateur.

Ces cinq mots — contrôleur, service, repository, entité, DTO — structurent tout le backend. Chacun a son chapitre dans ce document.

Les couches, dossier par dossier

```

backend/src/main/java/com/hooppicks/backendapplication/
├── BackendApplication.java    Le point de démarrage de l'application
├── entity/                   Les tables de la base, en objets Java
├── dto/                      Les objets échangés en JSON avec le frontend
├── repository/              L'accès à la base de données
├── controller/              Les URLs exposées (les "endpoints")
├── config/                  Configuration Spring, sécurité, gestion d'erreurs
├── security/                Sessions, mots de passe, vérification d'email
├── bet/                     Résolution des paris, statistiques communautaires
├── badge/                   Calcul des badges
├── league/                  Liges privées et fil d'activité
├── nba/                     Intégration balldontlie.io, Elo, calcul des cotes
├── espn/                    Intégration ESPN (effectifs, stats, classement)
├── news/                    Actualités NBA traduites en français
└── email/                   Envoi d'emails via l'API Brevo

```

Les technologies employées

Java et Spring Boot

Java est le langage. Il est *typé statiquement* : chaque variable a un type déclaré (`String`, `int`, `Match`), vérifié à la compilation. Une erreur de type est détectée avant même que le programme ne démarre, contrairement à JavaScript où elle n'apparaît qu'à l'exécution.

Spring Boot est le cadre applicatif (*framework*) qui prend en charge tout ce qui n'est pas propre à HoopPicks : démarrer un serveur web, router les URLs vers les bonnes méthodes, convertir les objets en JSON, ouvrir les connexions à la base, gérer les transactions. Le projet est sur la **version 4.1**, une version très récente — ce détail a des conséquences concrètes documentées plus loin (notamment le passage à Jackson 3, qui a causé un bug subtil lors de l'intégration ESPN).

Spring Boot fonctionne beaucoup par **annotations** : des marqueurs commençant par `@` posés sur les classes et les méthodes. `@RestController` signale « cette classe expose des URLs », `@Entity` signale « cette classe correspond à une table », `@Transactional` signale « cette méthode doit s'exécuter en une seule transaction de base de données ». Le framework les lit au démarrage et branche le comportement correspondant.

L'injection de dépendances

C'est le mécanisme central de Spring, et il est utilisé partout dans ce projet. Une classe ne crée pas elle-même les objets dont elle a besoin : elle les **reçoit en paramètres de son constructeur**, et Spring se charge de les lui fournir au démarrage.

```

@Service
public class BetResolutionService {
    private final BetRepository betRepository;

    // Spring appelle ce constructeur et passe l'objet lui-même.
    public BetResolutionService(BetRepository betRepository) {
        this.betRepository = betRepository;
    }
}

```

L'intérêt : la classe ne dépend que d'un type, jamais d'une construction concrète. Cela rend le code testable — dans les tests, on passe une fausse version du repository — et évite d'avoir à câbler la moitié de l'application à la main.

JPA, Hibernate et PostgreSQL

PostgreSQL est la base de données : c'est là que sont réellement stockés les utilisateurs, les paris, les matchs.

JPA (Jakarta Persistence API) est une norme qui permet de manipuler la base à travers des objets Java plutôt qu'en écrivant du SQL. **Hibernate** en est l'implémentation concrète utilisée ici. Concrètement : la classe `Match` correspond à la table `match`, chacun de ses champs à une colonne, et écrire `matchRepository.save(match)` génère automatiquement le `INSERT` ou l'`UPDATE` correspondant.

Cette commodité a un revers qui a été rencontré sur ce projet : quand on manipule une liste d'objets liés à d'autres objets, Hibernate peut émettre une requête SQL par élément sans qu'on s'en rende compte — c'est le fameux **problème N+1**. Il a été détecté lors des tests de charge (plus de 140 000 requêtes quasi identiques sur une table de 30 lignes) et corrigé ; le détail est documenté dans le chapitre sur les entités.

Lombok

Une bibliothèque qui génère automatiquement le code répétitif de Java. Au lieu d'écrire à la main un `getter` et un `setter` pour chacun des vingt champs d'une classe, on pose `@Getter` et `@Setter` sur la classe et Lombok les fabrique à la compilation. Les annotations rencontrées dans ce projet :

- `@Getter` / `@Setter` — génèrent les accesseurs de tous les champs.
- `@NoArgsConstructor` — génère un constructeur sans argument (JPA en a besoin).
- `@AllArgsConstructor` — génère un constructeur prenant tous les champs.

Le code généré n'apparaît pas dans les fichiers source. Si vous voyez `user.getUsername()` appelé alors qu'aucune méthode `getUsername()` n'est visible dans `User.java`, c'est Lombok.

Les décisions d'architecture à connaître

Certains choix de ce projet sont volontaires et contre-intuitifs. Les connaître évite de « corriger » quelque chose qui fonctionne comme prévu.

Les cotes sont fixes, il n'y a pas de marché dynamique. Sur un vrai site de paris, les cotes évoluent en permanence selon les mises. Ici elles sont calculées une seule fois lors de la synchronisation du match, stockées directement sur l'objet `Match`, et ne bougent plus jamais. C'est un choix assumé, pas un raccourci à combler.

Les sessions sont stockées en mémoire. Quand un utilisateur se connecte, sa session est conservée dans une simple structure en mémoire (`SessionStore`), pas en base ni dans Redis, et il n'y a pas de token JWT. Conséquence directe et acceptée : **redémarrer le backend déconnecte tout le monde**. À l'échelle de ce projet, c'est un compromis délibéré.

Il existe deux mécanismes d'administration distincts, volontairement séparés. Les routes `/admin/**` sont protégées par une clé statique passée dans un en-tête HTTP (`X-Admin-Key`), pensées pour un usage en ligne de commande et jamais appelées par le frontend. Les routes `/console/**` sont protégées par le cookie de session plus un drapeau `isAdmin` sur l'utilisateur, et alimentent la page `/admin` de l'interface. Les deux couvrent en partie les mêmes actions, avec des niveaux de traçabilité différents.

Il n'y a pas de rôle « super-administrateur ». N'importe quel administrateur peut rétrograder n'importe quel autre (sauf lui-même). Le pire cas est réversible via la clé statique, que seul le propriétaire du projet connaît.

Deux sources de données NBA cohabitent. L'API balldontlie.io fournit les matchs et les scores, mais son offre gratuite ne donne ni feuille de match détaillée, ni distinction entre joueur actif et retraité, ni classement officiel. ESPN a été ajouté en complément pour combler ces trois manques. Les deux systèmes n'ont aucun identifiant commun : le lien entre un match et son équivalent ESPN est reconstitué par date et sigles d'équipe.

Le classement Elo interne et le classement officiel NBA sont deux choses différentes. L'Elo est une note de force calculée par l'application à partir des résultats, et c'est **la seule donnée utilisée pour fixer les cotes**. Le classement officiel importé d'ESPN (victoires, défaites, série en cours) est purement informatif. Il ne faut ni les confondre ni les fusionner.

Conventions de lecture

Chaque fichier du projet est présenté selon le même schéma :

- 14. Un titre indiquant son chemin exact dans le projet.
- 15. Un paragraphe **Rôle** : à quoi il sert, quel problème il résout.
- 16. Le détail de ses champs et de ses méthodes, une par une.
- 17. Son **code source complet**, reproduit intégralement.

Les commentaires présents dans le code expliquent le **pourquoi** plutôt que le **quoi** — c'est une convention explicite du projet. Quand un commentaire du code justifie une décision, ce document la reprend et la développe.

Le document suit l'ordre des couches : d'abord les entités et les DTOs (les données), puis les contrôleurs et les repositories (les entrées et l'accès aux données), puis la configuration, la sécurité et la logique métier, et enfin les intégrations externes.